[RefactLab]
Refactoring Lab

Jan Corfixen Sørensen

Introduction: Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior. Its heart is a series of small behavior preserving transformations. Each transformation (called a "refactoring") does little, but a sequence of these transformations can produce a significant restructuring. Since each refactoring is small, it's less likely to go wrong. The system is kept fully working after each refactoring, reducing the chances that a system can get seriously broken during the restructuring.

Objectives:

- Identify and understand Bad Code Smells.
- Apply refactorings to get rid of Bad Code.

Classwork:

- Make sure you have your own feature branch, using *"git checkout -b your-feature development"*.
- Please follow the feature branch workflow [GitHub flow].
- Install [sonarlint].
- Find Code smells in JHotDraw based on your change request and sonarlint (shouldn't be a problem).
- Apply one or more suitable Refactoring Patterns to get rid of the bad code smells [1].

Portfolio Work:

- Describe the code smell that triggered your refactoring, see Chapter 4 in [Ker05]. Describe what you plan to change by refactoring. Describe the strategy of the refactorings. Which of the refactorings from [Ker05] did you apply and what was the reasoning behind it?
- Remember to describe the strategies and purpose of the Refactorings.

[1] See `http://refactoring.com/catalog/`